12_sysdesign_critic

Section 8: System Design Interview

This section treats Helios as a system to be defended on a whiteboard. A system-design interviewer does not want product trivia; they want to know whether you reason about scale, failure, observability, latency, and correctness as first-class concerns — and whether your design choices have *reasons*. For each question I state the general principle first, then show how a specific Helios component embodies it. The recurring theme: Helios is a *governed, deterministic control plane around a non-deterministic model*, and almost every system-design property falls out of that one decision.

Scalability

Q1. How does Helios scale from one funnel on one dataset to many tenants and many warehouses?

Ideal response. The general principle is to push variability behind a stable interface so the expensive parts (compute, governance, statistics) stay shared and only the *configuration* multiplies. A naive analytics tool hard-codes its SQL to a schema; scaling it to a new tenant means a rewrite. I designed Helios so the only thing that knows SQL dialect or schema is `warehouse-mcp`, and the only thing that knows a tenant's metrics is its `semantic_models.yml` registry. To add a tenant I add a registry + a byte budget + a credential; the seven agents, the FSM, and `stats-mcp` are unchanged because they operate on the typed "Finding" envelope, not on rows. To add a warehouse (Snowflake, Databricks, DuckDB) I add an adapter behind `warehouse-mcp`; because `semantic-mcp` is the sole SQL author, dialect differences are localized to one server and validated by the same `reconcile`-to-<=0.5% test on both engines. The failure mode this avoids is the classic "every customer is a fork." The tradeoff I accepted: a governed semantic layer is more upfront work than letting the model free-text SQL, but it is the thing that makes horizontal scale *and* trust possible at once.

Why Helios demonstrates it. Multi-tenancy = per-tenant `semantic_models.yml` (28 metrics + 16 dimensions today) + per-tenant byte budget; warehouse-agnosticism = adapters behind `warehouse-mcp` with `semantic-mcp` as the single SQL author.

Q2. The dimensional search space is combinatorial — 16 dimensions, many values each. How do you keep that from exploding?

Ideal response. Combinatorial search must be bounded by *value-of-information*, not enumerated. The general move is best-first search with hard depth/breadth caps and a pruning predicate. Diagnose treats RCA as best-first search over the dimensional space, always expanding the slice with the largest *rate* effect (genuine behavior change) before chasing *mix* effects (composition artifacts), and prunes any branch below `MIN_RATE_EFFECT`, with `significance_test` p > 0.05, or below the minimum sample. Search is breadth-bounded (`MAX_BRANCHING = 4`) and depth-bounded (`MAX_DEPTH = 3` dimensions). This is what keeps a 16-dimension space tractable in <5 min without exhaustively cross-tabulating millions of cells. The alternative — full cube materialization — would blow both the byte budget and the latency SLA and is exactly what a junior implementation does.

Why Helios demonstrates it. The Diagnose hypothesis-tree with `MAX_BRANCHING` / `MAX_DEPTH` caps and rate-effect-ordered expansion.

Q3. How do the MCP servers themselves scale?

Ideal response. Scale-out is easy when components are *stateless and idempotent*; it is a nightmare when they carry session state. I made `semantic-mcp`, `stats-mcp`, and `experiment-mcp` pure functions: same inputs -> same outputs, no internal state, so they can be replicated horizontally behind a pool and a crashed worker loses nothing. `warehouse-mcp` is the only stateful boundary (it holds the BigQuery client and the byte-budget counter), and even there the state of record is the `run_state` / `audit_log` tables in `helios_memory`, not process memory — so the server is effectively restartable. All cross-run learning lives in BigQuery + the vector store, never in an agent's heap. The failure mode avoided: "the analyst's laptop had the context." Determinism in `stats-mcp` (seeded scipy/statsmodels) also means horizontal replicas never disagree.

Why Helios demonstrates it. Stateless idempotent stats/semantic/experiment servers; durable state confined to `helios_memory` (BigQuery + vector store).

Reliability

Q4. What happens when a single agent or tool call fails mid-run? Do you ship a half-finished brief?

Ideal response. Reliability for an autonomous system means *fail closed, isolate the blast radius, and leave an audit trail* — never ship a partial result silently. The deterministic FSM is the backbone: the Orchestrator drives a finite-state machine (PLAN -> MONITOR -> DECOMPOSE -> DIAGNOSE -> CRITIC -> PRESCRIBE -> NARRATE -> END) and worker output never ships directly. Transient `warehouse-mcp`/BigQuery errors retry with exponential backoff (3 attempts). A single failed Diagnose branch is logged and pruned, not fatal. But if Monitor or Decompose fail entirely,

the Orchestrator *aborts* and writes an audit record — no partial brief ships. A clean (no-anomaly) run is not silence either: Narrator emits a "no material change" brief and still writes a `run_state` row, so absence-of-finding is itself recorded. The principle is that an autonomous system that runs unattended must make its non-actions auditable, or operators stop trusting it.

Why Helios demonstrates it. Orchestrator FSM with abort-on-upstream-failure, bounded retries, branch-level failure isolation, and a `run_state` / `audit_log` row for every run including clean ones.

Q5. The LLM is non-deterministic. How is this system reliable at all?

Ideal response. The trick is to make the model a *composer* inside a deterministic shell, never the control plane. The FSM decides flow; the LLM only chooses which governed tool calls to emit at a node. All math is deterministic Python in `stats-mcp` (seeded), so decomposition, significance, power, and forecast are reproducible byte-for-byte. All SQL is emitted by `semantic-mcp` from a governed registry, so two runs over the same data reference the same columns. The model's variance is therefore confined to *which hypotheses it explores*, and the Critic + significance gates + reconcile-to-<=0.5% absorb that variance before anything ships. This is the difference between "the AI said so" and a system whose every number is a tool output verbatim and whose every run is reconstructable from `audit_log`.

Why Helios demonstrates it. Deterministic FSM + deterministic `stats-mcp` + governed `semantic-mcp` confine non-determinism to hypothesis selection, which the Critic then filters.

Q6. How do you stop the system from re-alarming on the same known issue every single run?

Ideal response. An always-on diagnostic that cries wolf weekly gets muted, so reliability includes *not re-nagging*. Helios persists a suppression list in `helios_memory`: when a stakeholder acknowledges a finding (or the Critic auto-suppresses), a row is written with a `reason` and a TTL (default 30d, so a re-emerging issue eventually re-surfaces). Diagnose and Critic both consult it before promoting a leaf; a match is auto-DROPped with reason "suppressed." Known seasonality and launches live in `seasonality_calendar` / `launch_calendar`, which the Critic uses to refute "this is just the post-holiday dip." This is the operational maturity that separates a demo from something a team would actually leave running.

Why Helios demonstrates it. The memory suppression list (TTL'd) + seasonality/launch calendars consulted by Diagnose and Critic.

Observability

Q7. A run produces a wrong diagnosis. How do you debug it after the fact?

Ideal response. Observability for an autonomous reasoning system means *full reconstructability*: you must be able to replay exactly what the system saw and did. Every MCP call from any agent appends an `audit_log` row — `run_id`, `step_seq`, `agent`, `mcp_tool`, `args_hash` (sha256 of normalized args), the actual governed `sql_text`, `bytes_scanned`, `latency_ms`, and Critic `verdict`. The control plane wraps all tool invocations, so this is not best-effort logging — it is structurally complete. Combined with the pinned dbt model SHA and the semantic-registry hash, a run is fully reproducible: I can see every hypothesis explored, every query run, why each finding PASSed/DOWNGRADEd/DROPped, and replay it. The failure mode avoided is the unobservable LLM agent that you can only "re-prompt and hope."

Why Helios demonstrates it. The `audit_log` table (per-tool-call, with `sql_text` + `args_hash` + verdict) and pinned model/registry hashes.

Q8. How do you verify the "0 hallucinated columns / 100% governed SQL" claim in production, not just in the eval?

Ideal response. A claim you cannot continuously verify is marketing. Because `audit_log.sql_text` records the actual SQL run and every query must have come through `semantic-mcp`, I can run an AST check over `sql_text` against the registry + GA4 schema and prove provenance — any column not in the registry is a hard alarm. The eval harness runs this same `hallucination.py` scorer as a *hard zero gate* in CI; in production the same check runs over the audit log. The principle: the same invariant should be enforced offline (CI gate) and observed online (audit-log scan) with the same code, so there is no gap between what you test and what you ship.

Why Helios demonstrates it. Shared hallucination AST-check run as a CI hard-zero gate and as a production audit-log scan, both sourced from governed `sql_text`.

Q9. What operational signals would you put on a dashboard for Helios itself?

Ideal response. Observe the things that protect the SLAs and the trust pillars: per-run `bytes_scanned` vs the 5 GiB byte budget, per-run wall-clock vs the <5 min target, per-agent `latency_ms` and token cost (the Opus stages are the cost drivers), Critic verdict distribution (a spike in DROPs signals a Monitor/Decompose regression upstream), DOWNGRADE-to-PASS conversion after re-query, and the longitudinal eval scoreboard (top-1 accuracy, decomposition MAPE, hallucination rate). The `run_state` table already aggregates cost, timing, and finding counts per run; the rest are rollups of `audit_log`. Meta-point: I instrument the *trust* metrics, not just the infra metrics, because for this product correctness is the SLA.

Why Helios demonstrates it. `run_state` cost/timing/finding rollups + `audit_log` latency/verdict aggregations + the archived eval history table.

Latency

Q10. How do you hit <5 minutes per run when BigQuery scans are slow and the model is in the loop?

Ideal response. Latency and cost are the same lever here — bytes scanned. The general principle is to estimate before you spend and to bound the search. Every query is `dry_run`-cost-checked *before* `run_query`; if the estimated `bytes_scanned` would exceed the per-run byte budget the call is rejected and the agent must narrow the window or dimensions rather than blindly retry (rule G3). dbt marts are partitioned by `event_date` and clustered by `device_category, channel_group`, so partition/cluster pruning keeps scans small, and `fct_daily_funnel` is pre-aggregated so the hot path reads a day-grain table, not raw events. The hypothesis tree's `MAX_DEPTH` / `MAX_BRANCHING` caps bound how many queries a run can issue. Net effect: a representative full benchmark run scans ~2.1 GB and completes in ~4m12s, inside both the 5 GiB budget and the 5-minute SLA.

Why Helios demonstrates it. Mandatory `dry_run` byte gate (G3), partitioned/clustered marts, pre-aggregated `fct_daily_funnel`, and bounded hypothesis search.

Q11. The model itself adds latency. How do you keep token-time down without hurting accuracy?

Ideal response. Spend the expensive model only where the cost-of-error is high. I reserve Opus for the three reasoning-critical roles — Orchestrator (planning/budget/routing), Diagnose (combinatorial hypothesis-tree search), and Critic (adversarial refutation that must out-think Diagnose) — and use Sonnet for the bounded, mechanical stages: Monitor, Decompose, Prescribe, Narrator, where the action space is small (which series to test, which decomposition to run, which template to fill). I also compact context aggressively: each worker gets the run plan, the canonical catalog, and the upstream stage's *typed JSON* output — never row-level dumps. Result sets are summarized to aggregates before entering the model context, so token usage stays flat even as the dimensional search widens. The tradeoff: an all-Opus design is marginally more capable but breaks both the latency and the cost budget.

Why Helios demonstrates it. The Opus/Sonnet split by cost-of-error + compacted typed-JSON hand-offs that keep row data out of context.

Evaluation

Q12. How do you know any of this is correct, and how do you stop a regression from shipping?

Ideal response. Correctness for a system that emits causal claims must be *measured against ground truth*, continuously. The live GA4 export has real anomalies but they are unlabeled, so I built an offline labeled benchmark: synthetic anomalies are injected into a frozen copy of the data — a rate perturbation changes only `r_i`, a volume perturbation changes only `w_i` — and the pipeline must rediscover what was hidden, graded against the ground truth recorded at injection time. 50 labeled scenarios span 7 buckets (single/multi rate, single/multi mix, seasonality decoys, no-anomaly controls, data-quality artifacts). Headline contract: root-cause segment top-1 accuracy >=85% (vs <=45% for a naive "largest absolute segment delta" baseline), decomposition MAPE <=10%, and 0 hallucinated columns. This runs in GitHub Actions as a *required* check; CI fails the PR if any gate is breached or top-1 regresses more than the 2pt tolerance against the committed `main` baseline.

Why Helios demonstrates it. The offline injection benchmark (50 scenarios, perturbation primitives) + the CI gate in `eval/gates.yaml` with a regression tolerance.

Q13. The benchmark is synthetic. Doesn't that make the 85% number meaningless on real data?

Ideal response. Synthetic-injection is the *only* way to get labeled ground truth here, and I am explicit about its limits — that honesty is the point. The mechanism is principled: perturbations are applied to aggregates with conserved totals, so the analytic mix/rate/interaction split is *exact* and serves as the gold target for MAPE. The seasonality-decoy and data-quality buckets specifically test the failure modes that synthetic-only tests usually miss — the system must NOT flag a real seasonal swing and must catch a NULL spike or duplicated transaction_id as data quality, not behavior. So the benchmark stresses both detection and *refutation*. The honest caveat I state in interviews: this validates the decomposition-and-RCA machinery, not external causal validity, which is why Prescribe only *designs and sizes* experiments (the dataset is observational) and runs quasi-experimental readbacks rather than claiming live A/B causality.

Why Helios demonstrates it. Conserved-total perturbations (exact gold targets) + decoy/data-quality buckets + the stated observational-data limitation feeding quasi-experimental readbacks.

Q14. How do you keep the eval itself within budget — won't running 50 scenarios blow the cost ceiling?

Ideal response. The eval is a first-class consumer of the same cost controls as production. The harness runs every scenario through `warehouse-mcp.dry_run` first and aborts if total scanned bytes would exceed the per-run budget. To keep PR feedback fast and cheap, a smoke subset runs on every push and the full 50 run only on PRs to `main`. The harness pins random seeds, freezes the dbt model SHA, and records the registry hash, so a run is reproducible and the cost is stable across reruns. The principle: your test infrastructure must respect the same SLOs as

production, or it becomes the thing that's too expensive to run — and an eval you stop running is no eval at all.

Why Helios demonstrates it. `dry_run`-gated eval, push-time smoke subset vs PR-time full run, and seed/SHA/registry pinning for reproducible, bounded-cost runs.

Driving the Helios system-design whiteboard

A short live-sketch playbook. Draw it in this order — each layer answers a different interviewer concern, and the order *is* the argument.

1. **Start from the data flow (left to right).** GA4 events -> dbt staging (`stg_ga4__*`) -> intermediate (`int_ga4__sessionized` , `int_ga4__funnel_steps` with monotonic `reached_*` flags) -> marts (`fct_funnel` , `fct_daily_funnel` , `fct_orders` , `dim_*`). Say out loud: "session = `(user_pseudo_id, ga_session_id)`; the funnel is `session_start → view_item → add_to_cart → begin_checkout → add_shipping_info → add_payment_info → purchase`." This shows you can model data before you orchestrate agents.

2. **Draw the grounding boundary (the trust box).** Box the five MCP servers and label the two non-negotiable edges: `semantic-mcp` is the *only* path to SQL, `stats-mcp` is the *only* path to math. Put `warehouse-mcp` as the sole BigQuery client with the `dry_run → run_query` byte gate and `reconcile`-to-<=0.5%. This is where you earn "0 hallucinated columns" — say it explicitly: the model physically never holds a raw-SQL tool.

3. **Draw the FSM on top of the boundary.** The seven agents around a deterministic finite-state machine: Orchestrator (Opus) -> Monitor (Sonnet) -> Decompose (Sonnet) -> Diagnose (Opus) -> **Critic loop** (Opus, PASS/DOWNGRADE/DROP) -> Prescribe (Sonnet) -> Narrator (Sonnet). Draw the Critic as a *gate*, not a step. Emphasize: the LLM composes tool calls, the FSM controls flow. Drop in the decomposition identity `deltaR = mix + rate + interaction` as the centerpiece of the Decompose/Diagnose stage.

4. **Close with the eval loop and memory.** Off to the side, draw the offline benchmark feeding the CI gate (85% vs 45%, hard-zero hallucination, 2pt regression tolerance) and `helios_memory` (diagnosis history, suppression, calendars, action-tracking, `audit_log`) as the state plane the FSM reads/writes. End on the one-liner: "the hard part isn't generating insights — it's making them correct and trusted, and every box on this board exists to do exactly that."

Section 9: Tough Critic Questions

A skeptical interviewer is testing temperament as much as content: do you get defensive, or do you concede the valid part and then win on a specific design decision and a number? Every answer below does both. The meta-rule: never argue that the skepticism is wrong; argue that you *already designed for it*.

Q1. Why not just use dashboards?

The challenge. "Every company already has Looker or Tableau. Why build this?"

Ideal answer. Fair — dashboards are essential and Helios doesn't replace them. But a dashboard answers *what* happened; it cannot tell you *why*, *how many dollars* it costs, or *what to do*. When `session_conversion_rate` falls from 2.1% to 1.7%, the dashboard shows the line going down and then a human spends 1-3 analyst-days doing root-cause analysis by hand — and routinely confuses mix-shift with rate-change (Simpson's paradox). Helios is the *autonomous diagnostic layer that sits on top of the dashboard*: it runs `decompose_change` to separate composition change from behavior change, prices the movement in dollars, and ships a Decision Brief before anyone asks. The category isn't "BI"; it's autonomous growth diagnosis. A dashboard is a question generator; Helios is an answer generator that closes the insight-to-action gap from weeks to one review cycle.

Q2. Why not use ChatGPT (or a single LLM) directly on the data?

The challenge. "I could paste my schema into a chatbot and ask why conversion dropped. Why all this machinery?"

Ideal answer. You could, and you'd get a fluent, confident, frequently *wrong* answer — because a raw LLM hallucinates column names and computes statistics in token-space, which is not arithmetic. Helios structurally forbids both: the model never authors SQL (it composes governed metrics through `semantic-mcp`, yielding 0 hallucinated columns) and never does math in prose (decomposition, significance, power all run as deterministic Python in `stats-mcp`). A chatbot is also reactive — it answers one question at a time when prompted — whereas Helios runs on a schedule and proactively diagnoses before a human notices. The proof is the benchmark: the disciplined system hits 85% root-cause accuracy vs the 45% a naive aggregate read scores. The difference between those two numbers *is* the value of the machinery.

Q3. Why MCP? Isn't that just function-calling with extra steps?

The challenge. "MCP is a buzzword. You could do all this with plain tool/function calls."

Ideal answer. Mechanically, yes — MCP is a tool protocol. The reason it matters here is that it lets me make the trust boundary *structural* rather than *behavioral*. Each agent is granted only the MCP tools in its allow-list, so the Narrator, for instance, physically cannot call `run_query`; the model

can't break a rule it has no tool to break. That's stronger than "the system prompt says don't write SQL," which an LLM will eventually ignore. MCP also gives me clean capability isolation — `semantic-mcp` as sole SQL author, `stats-mcp` as sole math path — as stateless, independently testable, independently scalable servers. The win isn't the protocol; it's that capability-as-tool-boundary turns a policy into an invariant I can enforce and audit.

Q4. Why multiple agents instead of one?

The challenge. "Seven agents sounds like resume-driven complexity. One good agent could do this."

Ideal answer. The decisive reason is adversarial separation: the Critic must be a *different* agent than Diagnose, because an agent grading its own work is not a check. The Critic (Opus) attempts to refute every finding — mix-shift confound, insufficient sample, seasonality, data quality — and only PASSing findings ship; that's worth ~40 points of accuracy on the benchmark. Beyond that, the split lets me match model to cost-of-error (Opus on Orchestrator/Diagnose/Critic, Sonnet on the bounded stages) so I hit the cost and latency budgets, and it gives clean failure isolation in the FSM. I'd push back on "complexity": each agent has one narrow responsibility and a typed JSON hand-off — that's *less* coupled than one monolithic prompt trying to plan, query, decompose, diagnose, refute, and write all at once.

Q5. Why not just write one big SQL query?

The challenge. "A senior analyst could answer this with one well-written query. Why an AI system?"

Ideal answer. A senior analyst absolutely could — for *one* anomaly they already suspect, after they've context-switched in. Two problems. First, the hard part of root-cause is *search*: you don't know which of 16 dimensions and their crossings holds the cause, so a single static query can't express the best-first hypothesis-tree walk that drills rate-effects before mix-effects. Second, it doesn't scale or persist: the query lives in someone's head, runs when they're free, and forgets what it found last week. Helios runs unprompted on a schedule, bounds the combinatorial search (`MAX_DEPTH=3`, `MAX_BRANCHING=4`), keeps every scan inside a 5 GiB budget via `dry_run` gating, and remembers via `helios_memory`. The SQL still gets written — by `semantic-mcp`, governed and reconciled — just not by hand and not once.

Q6. Isn't this over-engineered for a portfolio project?

The challenge. "This is a 3-month static public dataset. Why the semantic layer, the FSM, the eval harness, the memory store?"

Ideal answer. I'd concede that for a one-off chart, yes, it would be. But the project's thesis is precisely that *the hard part of AI analytics is not generating insights — it's making them correct and trusted* — and you cannot demonstrate that with a thin demo. Each piece earns its place against a specific failure: the semantic layer kills hallucinated columns (0%), the deterministic stats layer kills token-space math, the Critic + eval kill confident-but-wrong diagnoses (85% vs 45%), and memory kills weekly re-nagging. I also scoped honestly: it's single-tenant batch today, with multi-tenant/streaming explicitly deferred to a later phase. Over-engineering is complexity without a corresponding failure it prevents; every component here maps to a named failure mode and a number. That mapping is the senior signal.

Q7. How do you KNOW the diagnoses are right?

The challenge. "You assert 85%. On what? The model could be confidently wrong and you'd never know."

Ideal answer. That's exactly why I don't trust it on assertion — I measure it against ground truth I control. I inject synthetic anomalies into a frozen copy of the data where I *know* the true root cause (a rate perturbation moves only `r_i`, a volume perturbation only `w_i`), then grade whether the pipeline rediscovers the hidden segment and effect. Across 50 labeled scenarios, top-1 root-cause segment accuracy is >=85% and decomposition MAPE <=10%, both gated in CI. On live data I can't have a label, so there I lean on the in-run checks: `reconcile`-to-<=0.5%, `significance_test` gating, and the adversarial Critic. "Right" is decomposed into measurable-offline plus verified-in-run; I never ask anyone to trust the model's word.

Q8. What happens when the LLM is wrong?

The challenge. "Models fail. When yours produces a bad hypothesis, what stops it from shipping?"

Ideal answer. I designed assuming the model *will* be wrong, and confine the damage to hypothesis *selection* — the one place non-determinism lives. A bad hypothesis hits three filters before it can ship: `significance_test` (p>0.05 prunes the branch), `reconcile` (>0.5% drift fails the finding), and the Critic (refutes confound/sample/seasonality/data-quality with verdict DROP). A plausible-but-thin finding is DOWNGRADEd and sent back to Diagnose for a targeted re-query, bounded by `MAX_REFUTE_ROUNDS=2`, after which it's demoted to a watchlist note rather than a confident finding. If an upstream stage fails outright, the FSM aborts and writes an audit row — it fails closed, never shipping a partial brief. And because every step is in `audit_log`, a wrong run is fully reconstructable for debugging.

Q9. Is 85% too low to trust?

The challenge. "85% means one in seven diagnoses is wrong. Why would I act on that?"

Ideal answer. Two honest framings. First, the comparison: the realistic human/naive baseline scores ~45% on the same benchmark — 85% is nearly a doubling, and the relevant bar is "better and faster than the status quo," which it clears decisively. Second, the *shape* of the error matters: top-3 accuracy is ~95%, so when the #1 segment is wrong the true cause is almost always in the shortlist the analyst reviews — and the Critic plus the dollar-at-risk ranking mean you're triaging the highest-value findings first, not acting blind on a single guess. Helios also doesn't claim final authority: it ships a *Decision Brief* a human reviews in <5 minutes with full evidence and an audit trail. So it's not "act on 85% blind"; it's "start every investigation 40 points ahead with the evidence already assembled." I'd take that trade every time.

Q10. Why should anyone trust the dollar figures?

The challenge. "Revenue-at-risk sounds like a number you made up to look impressive."

Ideal answer. It would be, if it were a model guess — so it isn't one. Revenue-at-risk is a deterministic computation from governed metrics: at a leaf finding it's (counterfactual rate at t0 - observed rate at t1) x affected `sessions` x `aov`, all canonical metrics pulled via `semantic-mcp` and reconciled to source within 0.5%. The LLM supplies no digits; the number is a `stats-mcp` /governed-SQL output verbatim, traceable through `audit_log` to the exact query. I also validate the *method* in the eval: the injector computes a true `dollar_at_risk_usd` from the conserved-total perturbation, and Helios's estimate is graded against it with a <=15% error target. So the dollar figure is auditable, reconciled, and benchmarked — three things a made-up number is not. The honest caveat: it's a correlational revenue-at-risk on observational data, not a causal lift estimate, which is why Prescribe pairs it with a powered experiment to confirm.

Q11. What does a run cost, and does it actually scale?

The challenge. "BigQuery and Opus tokens aren't free. What's the unit economics, and what breaks at scale?"

Ideal answer. Per run, BigQuery cost is capped *structurally*: every query is `dry_run`-estimated before execution and rejected if it would exceed the 5 GiB byte budget, and a representative full run scans ~2.1 GB in ~4m12s — well inside budget and the <5 min SLA. Token cost is controlled by the Opus/Sonnet split (Opus only on the three reasoning-critical roles) and by compacting context to typed JSON aggregates so tokens stay flat as the search widens. On scale: the stats/semantic/experiment servers are stateless and idempotent, so they replicate horizontally; durable state is in BigQuery, not process memory; and adding a tenant is adding a registry + budget + credential, not a fork. What I'd watch as the real scaling cost is the Opus reasoning stages and per-tenant BigQuery scan — both observable in `run_state` / `audit_log`, both bounded by the byte budget and the search caps. The honest limit: it's single-tenant batch today; multi-tenant and streaming are a deliberately deferred phase, not hand-waved.

Q12. Why not just buy Amplitude or Mixpanel?

The challenge. "Product-analytics tools already do funnels and retention. Why not use one?"

Ideal answer. They're excellent and I'd happily sit Helios alongside one. But Amplitude and Mixpanel are still *descriptive*: they show the funnel and let a human pull a report — they answer *what* and some *who*, on human trigger. None of them autonomously decomposes a movement into mix-shift vs rate-change to dissolve Simpson's paradox, prices it in dollars, refutes it adversarially, and prescribes a powered experiment — unprompted, on a schedule, grounded in governed SQL with an offline accuracy benchmark. That diagnostic-and-prescriptive loop is the category Helios creates, and it's complementary: you could feed its briefs from a warehouse that also serves Mixpanel. The buy-vs-build framing also misses the portfolio point — the engineering thesis is the *trust architecture* (governed SQL + deterministic stats + adversarial eval), which is exactly the part a SaaS tool hides and an interviewer wants to see me reason about.